

TEK-UP University

École d'Ingénierie & Université privée

Société : Gérance Informatique

Rapport de stage

VOC Platform

Vulnerability Operations Center

Plateforme automatisée de gestion des vulnérabilités

Réalisé par	TBINI Mustapha Amin
Encadrement	Mr. MDAOUKHI Adel
Société d'accueil	Gérance Informatique
Université	TEK-UP University
Année universitaire	2025–2026
Lieu de déploiement	192.168.184.135 (VMware)
Période	Stage universitaire – année 2025–2026

Remerciements

Au terme de ce stage, je tiens à exprimer ma gratitude à toutes les personnes qui ont contribué à la réalisation de ce projet.

Je remercie tout d'abord mon encadrant, **Mr. MDAOUKHI Adel**, pour son accompagnement, ses conseils techniques et la confiance qu'il m'a accordée tout au long de ce travail.

Je remercie également la société **Gérence Informatique** pour son accueil, ainsi que l'ensemble de ses équipes pour leur disponibilité et leur entraide.

Enfin, je remercie l'université **TEK-UP University** et son corps enseignant pour la qualité de la formation dispensée durant l'année universitaire 2025–2026.

Table des matières

Remerciements	1
1 Introduction générale	3
1.1 Contexte	3
1.2 Présentation du projet	3
1.3 Organisation du rapport	3
2 Problématique et solution	3
2.1 Le constat (problème)	4
2.2 La solution apportée	4
2.3 Objectifs du projet	4
3 Environnement de stage	5
3.1 Université TEK-UP University	5
3.2 Société Gérance Informatique	5
3.3 Encadrement	5
3.4 Méthodologie suivie	5
4 Outils et technologies	5
4.1 Conteneurisation et déploiement	6
4.2 Développement	6
4.3 Ordonnancement et files de tâches	6
4.4 Analyse de vulnérabilités	7
4.5 Intelligence sur les menaces	7
4.6 Stockage et visualisation	8
4.7 Gestion des agents	8
4.8 Gestion des services informatiques	9
4.9 Infrastructure et versionnement	9
4.10 Tableau récapitulatif	9
5 Architecture générale	10
5.1 Vue d'ensemble	10
5.2 Gestion centralisée des agents (Fleet)	10
5.3 Services et ports	10
5.4 Topologie réseau	11
5.5 Structure du dépôt	11

6	Modélisation UML	13
6.1	Diagramme de cas d'utilisation	13
6.2	Diagramme de classes	13
6.3	Diagramme de séquence	13
6.4	Diagramme d'états	13
6.5	Diagramme de déploiement	14
7	Pipeline de gestion des vulnérabilités	16
7.1	Déroulement général	16
7.2	Description des étapes	16
7.3	Analyse différentielle	16
7.4	Synchronisation GLPI → ELK	18
7.5	Ordonnancement et files	18
8	Moteur de calcul du risque	19
8.1	Entrées	19
8.2	Algorithme de notation	19
8.3	Schéma de décision	19
8.4	Exemples de calcul	19
8.5	Règles de création des tickets GLPI	21
9	Modèle de données et stockage	22
9.1	Indices Elasticsearch	22
9.2	Vues de données Kibana	22
9.3	Exemple de document de vulnérabilité	22
10	Sécurité de la plateforme	24
10.1	Chiffrement de Kibana	24
10.2	Authentification	24
10.3	Isolation des conteneurs	24
10.4	Supervision des agents	24
10.5	Transport des données	24
11	État opérationnel du déploiement	25
11.1	Santé de la plateforme	25
11.2	Preuves de l'extension du scan	25
11.3	Répartition des données (indice du 08.15)	25
11.4	Ports les plus fréquents	25
11.5	Agents Fleet	26

12 Difficultés rencontrées et solutions apportées	27
12.1 Serveur Fleet instable	27
12.2 Erreurs de chiffrement de Kibana	27
12.3 Agent externe sans flux de données	27
12.4 Stabilité du worker	27
12.5 Santé d'Elasticsearch	27
12.6 Scan partiel	27
12.7 Tickets dupliqués	27
13 Tests et validation	28
13.1 Tests unitaires	28
13.2 Validation de bout en bout	28
14 Conclusion générale	29
A Annexe A : variables d'environnement principales	30
B Annexe B : opérations courantes	30
C Annexe C : tests	30
D Annexe D : glossaire	31

1 Introduction générale

1.1 Contexte

La sécurité des systèmes d'information est devenue un enjeu stratégique pour toutes les organisations. Chaque année, des dizaines de milliers de vulnérabilités sont publiées dans les bases de référence (CVE), et un nombre croissant d'entre elles sont activement exploitées avant même la disponibilité d'un correctif. Le catalogue CISA KEV recense les vulnérabilités déjà exploitées dans la nature, et les outils d'analyse estiment que seule une fraction des CVE constitue un risque réel pour une organisation donnée.

Face à ce constat, la simple collecte de vulnérabilités ne suffit plus. Les équipes de sécurité doivent être capables de :

- **prioriser** efficacement les failles à traiter, en fonction du risque réel pour l'activité ;
- **corrélér** les découvertes avec le contexte des menaces (exploits publics, exploitation active, indicateurs de compromission) ;
- **orchestrer** le traitement : ouverture de tickets, suivi de la remédiation, mesure de l'amélioration.

1.2 Présentation du projet

Le projet développé durant ce stage, la **VOC Platform** (*Vulnerability Operations Center*), est une plateforme automatisée de gestion des vulnérabilités qui :

- scanne périodiquement le réseau interne (**tous les ports** TCP, avec détection des services et des systèmes d'exploitation) ;
- enrichit chaque découverte avec de l'**intelligence sur les menaces** (NVD, MISP, EPSS, CISA KEV, OSV, Exploit-DB) ;
- calcule un **score de risque** orienté métier entre 0 et 10 ;
- publie les résultats dans une **stack de visualisation** (Elasticsearch + Kibana) ;
- ouvre automatiquement des **tickets de remédiation** dans un outil ITSM (GLPI) ;
- partage les nouvelles découvertes sous forme d'**événements MISP** ;
- supervise l'ensemble via une **observabilité unifiée** (Fleet + Beats).

1.3 Organisation du rapport

Ce rapport est organisé comme suit :

1. la **problématique** à laquelle la plateforme apporte une solution, puis la présentation de l'**environnement de stage** ;
2. la présentation des **outils et technologies** employés et les raisons de leur choix ;
3. l'**architecture** de la plateforme et sa **modélisation UML** (cas d'utilisation, classes, séquence, états, déploiement) ;
4. le **pipeline** de traitement des vulnérabilités et le **moteur de calcul du risque** ;
5. le **modèle de données**, la **sécurité**, l'**état opérationnel**, les **difficultés** rencontrées et les **tests** ;
6. enfin, la **conclusion** générale du stage.

2 Problématique et solution

2.1 Le constat (problème)

Les équipes en charge de la sécurité des systèmes d'information font face à plusieurs difficultés récurrentes :

- **Scans manuels et ponctuels** : les audits de vulnérabilités sont réalisés de manière ponctuelle et artisanale, avec des outils disparates et des périmètres partiels (liste fixe de ports, fréquence irrégulière).
- **Absence de vision centralisée** : les résultats sont éparpillés dans des fichiers et aucun indicateur global n'est disponible pour piloter la sécurité de l'infrastructure.
- **Pas de corrélation avec les menaces** : une faille ancienne à fort impact est traitée de la même manière qu'une faille récemment exploitée dans la nature ; aucun contexte de menace (EPSS, catalogue KEV, flux d'intelligence MISP) n'est pris en compte.
- **Notation de risque incohérente** : la seule note CVSS ne reflète pas l'exposition réelle de l'entreprise (actif critique, exploit public, exposition réseau, etc.).
- **Suivi de correction manuel** : l'ouverture et le suivi des tickets de remédiation sont effectués à la main, ce qui génère des oublis et des délais.
- **Observabilité hétérogène** : les journaux, les métriques et la disponibilité des services sont suivis sans outil commun ni historique centralisé.

2.2 La solution apportée

La **VOC Platform** automatise l'ensemble de ce cycle de vie :

- **Scan automatique et complet** : découverte du réseau (`nmap -sn`), scan de **tous les ports TCP** (65535 ports, `-p-`), détection des versions de services (`-sV`) et **empreinte du système d'exploitation** (`-O`).
- **Analyse différentielle** : chaque résultat est comparé à l'état précédent (memorisé dans Redis) pour distinguer les vulnérabilités *nouvelles*, *toujours présentes* ou *résolues*.
- **Enrichissement et corrélation** : corrélation avec le flux CVE de la NVD, recherche d'IOC dans MISP, ajout du score EPSS, de la présence dans le catalogue CISA KEV, d'Exploit-DB et d'OSV.
- **Score de risque métier** : un moteur de risque dédié (FastAPI) combine CVSS, criticité de l'actif, activité de menace, exploitabilité, exposition réseau, EPSS et KEV pour produire une note normalisée entre 0 et 10.
- **Visualisation centralisée** : tous les résultats sont indexés dans Elasticsearch et consultables dans Kibana via des tableaux de bord.
- **Circuit ITSM automatisé** : toute vulnérabilité dont le risque est supérieur ou égal à 7 génère automatiquement un ticket dans GLPI.
- **Partage de renseignements** : les nouvelles vulnérabilités sont publiées sous forme d'événements MISP.
- **Observabilité unifiée** : les agents Elastic (Fleet et Beats) collectent les journaux, les métriques, les flux réseau, les traces d'audit et la disponibilité des services.

2.3 Objectifs du projet

Les objectifs fixés au début du stage étaient les suivants :

1. déployer une plateforme de scan automatique fiable et complète ;
2. assurer la corrélation des découvertes avec les sources de menaces ;
3. produire un score de risque orienté métier, reproductible et explicable ;

4. offrir une visibilité centralisée (tableaux de bord) ;
5. automatiser l'ouverture et le suivi des tickets de remédiation ;
6. superviser la plateforme et les agents déployés.

3 Environnement de stage

3.1 Université TEK-UP University

TEK-UP University est un établissement d'enseignement supérieur délivrant des formations d'ingénierie dans les domaines des technologies de l'information et de la communication. Le présent stage s'inscrit dans le cadre de l'année universitaire 2025–2026 et du programme de formation d'ingénierie.

3.2 Société Gérence Informatique

Gérence Informatique est une société spécialisée dans les services liés aux systèmes d'information : administration de réseaux et de serveurs, gestion des parcs informatiques et sécurité des systèmes. C'est dans ce contexte que le besoin d'une plateforme centralisée de gestion des vulnérabilités a été identifié, afin de professionnaliser la surveillance sécuritaire de l'infrastructure.

3.3 Encadrement

Le stage a été encadré par **Mr. MDAOUKHI Adel**, qui a assuré le suivi technique du projet, la validation des choix d'architecture et la revue des développements réalisés.

3.4 Méthodologie suivie

Le travail a été conduit selon une démarche itérative :

1. **Analyse des besoins** : recensement des difficultés, des périmètres réseau et des outils déjà présents sur l'infrastructure ;
2. **Conception** : choix de la stack technique, définition de l'architecture et du pipeline de traitement ;
3. **Développement** : réalisation des modules (scan, enrichissement, score, indexation, tickets), des diagrammes et des tests unitaires ;
4. **Déploiement et intégration** : mise en place des services Docker, configuration des agents Elastic, résolution des problèmes d'exploitation ;
5. **Validation et exploitation** : vérification de bout en bout, analyse des résultats et suivi opérationnel.

4 Outils et technologies

Cette section décrit les outils et technologies utilisés dans le projet, ainsi que les raisons de leur choix. Les logos illustrent les technologies officielles employées.

4.1 Conteneurisation et déploiement



Docker

Description : Docker est une plateforme de conteneurisation qui permet d'embarquer une application et ses dépendances dans des conteneurs isolés et reproductibles. Chaque conteneur dispose de son propre espace processus, réseau et système de fichiers.

Pourquoi ce choix : La plateforme est composée de plus de dix-huit services interdépendants (bases de données, courtiers, applications, agents de collecte). Docker garantit un déploiement identique sur toute machine, simplifie la gestion des versions et accélère les redéploiements.



Docker Compose

Description : Docker Compose permet de définir et d'orchestrer un ensemble de conteneurs multi-services à partir d'un unique fichier YAML (réseaux, volumes, limites mémoire, variables d'environnement, ordre de démarrage).

Pourquoi ce choix : Le fichier `docker-compose.yml` centralise la définition des 18 services applicatifs plus le serveur Fleet. C'est le point d'entrée unique pour déployer et administrer l'ensemble de la plateforme.

4.2 Développement



Python 3.11

Description : Python est un langage de programmation généraliste, interprété et riche d'un vaste écosystème de bibliothèques (parsing, HTTP, tests, calcul).

Pourquoi ce choix : Les modules du pipeline (scan, enrichissement, score, indexation, tickets) sont écrits en Python : il offre une syntaxe lisible, une intégration native avec `python-nmap`, `requests` et `pytest`, et accélère fortement le développement des clients d'API (MISP, GLPI, NVD, etc.).



FastAPI

Description : FastAPI est un framework Python moderne et performant pour la construction d'API REST, basé sur Pydantic pour la validation des données et générant automatiquement une documentation OpenAPI.

Pourquoi ce choix : Le moteur de risque expose une API `POST /score` ainsi qu'une sonde de santé `GET /health`. FastAPI a été choisi pour sa vitesse d'exécution, la validation automatique des entrées, la documentation interactive et sa faible latence.

4.3 Ordonnancement et files de tâches



Celery et Celery Beat

Description : Celery est un système d'exécution de tâches asynchrones distribuées ; Celery Beat est son planificateur périodique (crontab).

Pourquoi ce choix : Le pipeline est organisé en tâches asynchrones (`scan`, `enrich`, `enhance`, `score`, `index_to_elk`, `create_ticket`) exécutées par des workers parallèles. Celery gère la répartition, les nouvelles tentatives et le suivi des résultats, tandis que Beat déclenche le scan au démarrage et toutes les 6 heures, ainsi que la synchronisation GLPI chaque heure.



RabbitMQ

Description : RabbitMQ est un courtier de messages implémentant le protocole AMQP (Advanced Message Queuing Protocol).

Pourquoi ce choix : Il sert de brique centralisatrice du pipeline : chaque famille de tâches possède sa file dédiée (`scan`, `enrich`, `score`, `index`, `ticket`), ce qui permet de paralléliser le traitement et d'absorber les pics de charge générés par les scans.



Redis

Description : Redis est une base de données clé-valeur en mémoire, très rapide et compatible avec la plupart des langages.

Pourquoi ce choix : Redis joue deux rôles : conserver l'état différentiel des scans (`voc:scan:state`) et servir de backend de résultats Celery. Sa rapidité est essentielle pour comparer à chaque cycle des milliers de vulnérabilités sans pénaliser le pipeline.

4.4 Analyse de vulnérabilités



nmap

Description : nmap est l'outil de référence pour la découverte de réseau, l'audit de ports et la reconnaissance de services et de systèmes d'exploitation.

Pourquoi ce choix : Il est utilisé avec les options `-sS -sV -O -p- -T4` pour scanner l'intégralité des 65535 ports TCP, identifier les versions des services et reconnaître le système d'exploitation des hôtes. L'accès aux bibliothèques Python (`python-nmap`) permet de l'intégrer nativement dans les tâches Celery.

4.5 Intelligence sur les menaces



MISP

Description : MISP (*Malware Information Sharing Platform*) est une plateforme de partage d'intelligence sur les menaces : événements, attributs (IOC), taxonomies et corrélations.

Pourquoi ce choix : La plateforme y recherche des indicateurs de compromission liés à chaque CVE et y publie, à l'inverse, les nouvelles vulnérabilités sous forme d'événements. C'est le pivot du volet *renseignement* du pipeline.



NVD

Description : La *National Vulnerability Database* est la base de référence des CVE, gérée par le NIST.

Pourquoi ce choix : L'API NVD fournit les méta-données de chaque CVE (score CVSS, CWE, références, versions impactées) qui servent de socle à la corrélation produit/version → CVE.



EPSS

Description : L'*Exploit Prediction Scoring System* (FIRST) estime, avec un score entre 0 et 1, la probabilité qu'une vulnérabilité soit exploitée dans les 30 jours.

Pourquoi ce choix : Le score EPSS est une entrée du moteur de risque : il permet de prioriser les failles réellement exploitables plutôt que de traiter uniquement par note CVSS.



CISA KEV

Description : Le catalogue KEV (*Known Exploited Vulnerabilities*) de la CISA recense les vulnérabilités activement exploitées par des acteurs malveillants.

Pourquoi ce choix : Toute CVE présente dans ce catalogue reçoit un bonus de risque (2.0 par défaut), car une exploitation en cours représente une menace immédiate pour le réseau.



OSV.dev & Exploit-DB

Description : OSV.dev agrège des vulnérabilités open-source ; Exploit-DB indexe les exploits publics disponibles.

Pourquoi ce choix : Ces deux sources complètent l'enrichissement : confirmation de l'existence d'un exploit public (`exploit_available`) et données supplémentaires pour les composants open-source.

4.6 Stockage et visualisation



Elasticsearch

Description : Elasticsearch est un moteur de recherche et d'analyse distribué basé sur Lucene, avec indexation journalière et agrégations en temps réel.

Pourquoi ce choix : Il stocke tous les documents de vulnérabilités (indices `vulnerabilities-*`), les données d'observabilité des Beats et l'état des tickets (`glpi-*`). Sa rapidité et ses agrégations permettent des tableaux de bord synthétiques.



Logstash

Description : Logstash est un pipeline de traitement de données qui collecte, transforme et achemine les événements vers Elasticsearch.

Pourquoi ce choix : Il reçoit les documents en HTTP (:5044) et les achemine vers les indices journaliers. Il centralise ainsi l'ingestion et facilite les transformations futures sans modifier les workers.



Kibana

Description : Kibana est l'interface de visualisation de la stack Elastic : tableaux de bord, vues de données, découvertes et administration de Fleet.

Pourquoi ce choix : Il fournit les tableaux de bord de vulnérabilités, les vues de données (Filebeat, Packetbeat, Metricbeat, Heartbeat, GLPI) et l'administration de Fleet. L'image a été personnalisée pour injecter les clés de chiffrement à l'initialisation.

4.7 Gestion des agents



Fleet Server et Elastic Agent

Description : Fleet Server est le point d' enrôlement central des Elastic Agents ; il distribue les politiques et les intégrations de collecte.

Pourquoi ce choix : Fleet permet d' enrôler des agents sur des hôtes externes (Linux/Windows) via une politique par type d'hôte et de superviser leur état (`online/degraded`). C'est la brique de *gouvernance des agents*.



Les Beats (Filebeat, Metricbeat, Packetbeat, Auditbeat, Heartbeat)

Description : Les Beats sont des agents légers de collecte de données pour la stack Elastic.

Pourquoi ce choix : Chaque Beat couvre un besoin d'observabilité : journaux (Filebeat), métriques système et Docker (Metricbeat), flux réseau (Packetbeat), traces d'audit (Auditbeat) et disponibilité des services (Heartbeat). Ils alimentent les tableaux de bord de supervision.

4.8 Gestion des services informatiques



GLPI

Description : GLPI est une solution ITSM open-source de gestion des parcs informatiques, des incidents et des demandes, dotée d'une API REST.

Pourquoi ce choix : C'est le circuit de remédiation : le pipeline y crée automatiquement un ticket pour chaque vulnérabilité dont le score de risque est ≥ 7 , avec une priorité et une liste de vérifications (*checklist*) adaptées à la sévérité.

4.9 Infrastructure et versionnement



Ubuntu Server (VMware)

Description : Ubuntu Server est le système d'exploitation hôte de la plateforme, hébergé sur une machine virtuelle VMware (192.168.184.135).

Pourquoi ce choix : Il offre un écosystème stable, des paquets récents pour Docker et les outils réseau (dont nmap), ainsi qu'une documentation abondante.



Git

Description : Git est un système de contrôle de version distribué.

Pourquoi ce choix : L'ensemble du code (docker-compose, workers, configuration des agents, rapport) est versionné dans un dépôt Git local, ce qui garantit la traçabilité des modifications et la reproductibilité du déploiement.

4.10 Tableau récapitulatif

Couche	Technologie	Rôle
Orchestration	Celery + Celery Beat	Pipeline de tâches asynchrones et ordonnancement
Courtier	RabbitMQ 3.12	Files de messages distribuées
État/cache	Redis 7	État de scan, backend de résultats, cache
Scanner	nmap (python-nmap)	Découverte + scan complet + OS
Intel menaces	MISP	Partage et enrichissement d'événements
Flux CVE	NVD API	Corrélation produit → CVE
Contexte exploit	EPSS, CISA KEV, OSV, Exploit-DB	Probabilité d'exploitation et exploits
Moteur de risque	FastAPI	Calcul du score de risque
ITSM	GLPI	Tickets de remédiation et gestion de parc
Recherche/viz	Elasticsearch + Kibana 8.11	Indexation, tableaux de bord, alertes
Ingestion	Logstash 8.11	Pipeline HTTP → Elasticsearch
Agents	Fleet Server / Elastic Agent / Beats	Collecte et gestion des agents

5 Architecture générale

5.1 Vue d'ensemble

La plateforme fonctionne comme un ensemble de services Docker interconnectés : un pipeline Celery (couplé à RabbitMQ et Redis) scanne le réseau, enrichit et note les vulnérabilités ; les résultats sont indexés dans la stack ELK et les tickets sont ouverts dans GLPI. La gestion des agents est assurée par un serveur Fleet. Le schéma suivant résume les composants et leurs relations :

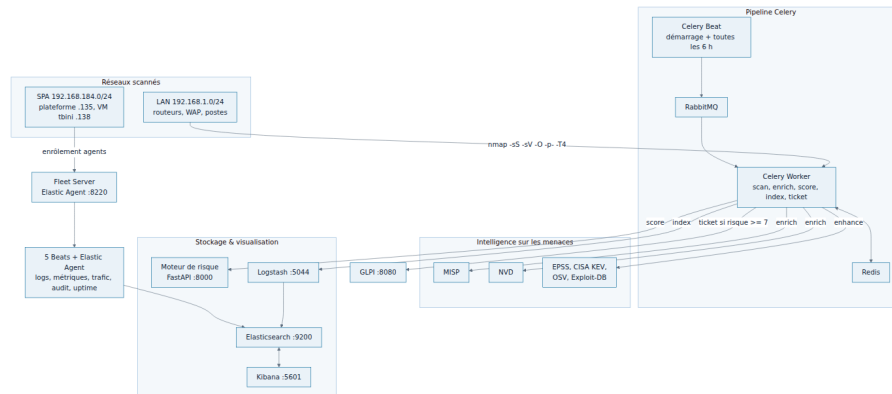


Figure 1 – Architecture générale de la plateforme VOC

5.2 Gestion centralisée des agents (Fleet)

Une des évolutions majeures du projet a été l'ajout du serveur **Fleet Server** (Elastic Agent dédié, port :8220). Il permet :

- d'enrôler des **agents externes** (hôtes Linux/Windows hors Docker) avec des politiques dédiées (`linux_hosts`, `windows_hosts`) ;
- de centraliser la configuration des collecteurs (intégrations système, logs, métriques) ;
- de superviser l'état de chaque agent depuis Kibana.

L'image Kibana a été personnalisée pour injecter les clés de chiffrement nécessaires (`encryptedSavedObjects`, `security` et `reporting`), ce qui a éliminé les erreurs de démarrage et les écrans de connexion fonctionnellement bloqués.

5.3 Services et ports

Service	Conteneur	Image	Port
RabbitMQ	voc-rabbitmq	rabbitmq:3.12-management-alpine	5672, 15672
Redis	voc-redis	redis:7-alpine	6379
Elasticsearch	voc-elasticsearch	elasticsearch:8.11.0	9200
Logstash	voc-logstash	logstash:8.11.0	5044, 9600
Kibana	voc-kibana	kibana:8.11.0 (custom)	5601
Fleet Server	voc-fleet-server	elastic-agent:8.11.0	8220
MariaDB (GLPI)	voc-mariadb	mariadb:10.6	3306
MariaDB (MISP)	voc-misp-db	mariadb:10.6	—
Moteur de risque	voc-risk-engine	custom (FastAPI)	8000
GLPI	voc-glpi	diouxx/glpi:latest	8080
MISP	voc-misp	ghcr.io/misp/misp-docker/misp-core	8443

Service	Conteneur	Image	Port
Worker Celery	voc-celery-worker	custom (Python + nmap, root)	–
Beat Celery	voc-celery-beat	custom (Python + nmap)	–
Filebeat	voc-elastic-agent	filebeat:8.11.0	–
Packetbeat	voc-packetbeat	packetbeat:8.11.0	– (hôte)
Auditbeat	voc-auditbeat	auditbeat:8.11.0	–
Metricbeat	voc-metricbeat	metricbeat:8.11.0	–
Heartbeat	voc-heartbeat	heartbeat:8.11.0	– (hôte)

5.4 Topologie réseau

Réseau	Périmètre
192.168.184.0/24	Réseau SPA : hôte plateforme .135 et agents externes
192.168.1.0/24	Réseau LAN : hôtes scannés et découverts (routeurs, WAP, postes)
172.18.0.0/16	Réseau bridge Docker des services
ens33	192.168.184.135 (interface principale)
ens34	192.168.87.3 (interface secondaire)

5.5 Structure du dépôt

Le projet est organisé dans le répertoire `/opt/voc-platform` :

Listing 1 – Structure du dépôt

```

1 /opt/voc-platform/
2 |-- docker-compose.yml          # 18 services + fleet-server
3 |-- .env                        # Secrets et configuration
4 |-- .env.example                # Gabarit (commitable)
5 |-- README.md                  # Vue d'ensemble
6 |-- PROJECT_REFERENCE.md        # R\ref\erence technique
7 |-- elasticsearch/             # (configuration via docker-compose)
8 |-- fleet-server/              # Configuration du serveur Fleet
9 |-- kibana/
10 |   |-- kibana.yml              # Configuration + cl\es de chiffrement
11 |   |-- Dockerfile              # Entrypoint personnalis\e
12 |   '-- dashboards/             # Tableaux de bord import\es via API
13 |-- logstash/
14 |   '-- pipeline/voc.conf        # Input HTTP -> output Elasticsearch
15 |-- risk-engine/
16 |   |-- main.py                 # FastAPI POST /score, GET /health
17 |   '-- requirements.txt
18 |-- scripts/
19 |   |-- *beat-config.yml         # Configurations des 5 Beats
20 |   |-- elastic-agent-config.yml
21 |   |-- icmp-monitor.sh         # Moniteur de ping ICMP (cron)
22 |   |-- init-es-users.sh
23 |   |-- misp-config.php
24 |   '-- install-windows-agent.ps1
25 |-- workers/
26 |   |-- Dockerfile              # python:3.11 + nmap (USER root)
27 |   |-- entrypoint-beat.sh      # Scan au d'emarrage + lancement beat
28 |   |-- tasks.py                # Application Celery et t\aches
29 |   |-- glpi_client.py          # Client REST GLPI

```

```
30 |   |-- glpi_sync.py           # Synchronisation GLPI -> ELK
31 |   |-- logstash_client.py     # Indexation ELK (incl. resolved)
32 |   |-- misp_client.py         # Recherche/cr\'eation d\'ev\'enements
33 |   |-- nvd_client.py         # Recherche CVE NVD
34 |   |-- ticket_enrichment.py   # Classification, ATT&CK, CIS, checklist
35 |   |-- threat_intel.py        # EPSS, KEV, OSV, Exploit-DB
36 |   |-- backfill_enrich.py     # R\'e-enrichissement historique
37 |   |-- knowledge/            # Bases de connaissances (CWE, ATT&CK, CIS
38 |   )                          # Tests unitaires
39 |   |-- tests/                # Rapport + diagrammes + ic\'^ones
   |-- reports/
```

6 Modélisation UML

La modélisation UML permet de décrire les différents aspects de la plateforme : les interactions utilisateurs, la structure logicielle, les échanges entre composants et le cycle de vie des données.

6.1 Diagramme de cas d'utilisation

Le diagramme de cas d'utilisation ci-dessous présente les acteurs et les fonctionnalités principales de la plateforme :

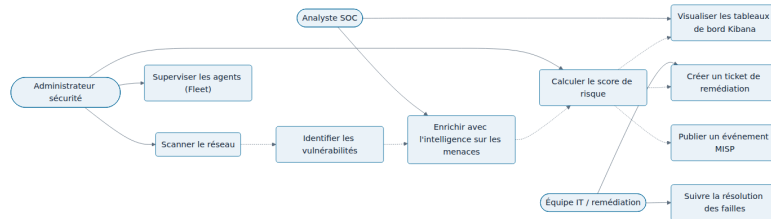


Figure 2 – Diagramme de cas d'utilisation de la plateforme VOC

Les acteurs identifiés sont :

- **Administrateur sécurité** : déclenche les scans, valide les scores et administre les agents via Fleet ;
- **Équipe IT / remédiation** : traite les tickets ouverts et confirme la résolution ;
- **Analyste SOC** : consulte les enrichissements et les tableaux de bord.

6.2 Diagramme de classes

Le diagramme de classes présente la structure du module des workers Celery, avec les principales classes et leurs relations :

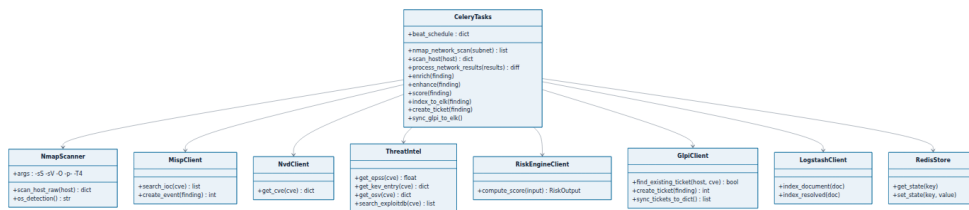


Figure 3 – Diagramme de classes du module workers

La classe centrale `CeleryTasks` orchestre les tâches du pipeline et délègue aux clients externes (`MispClient`, `NvdClient`, `ThreatIntel`, `RiskEngineClient`, `GlpiClient`, `LogstashClient`) ainsi qu'à la brique de scan (`NmapScanner`) et au stockage d'état (`RedisStore`).

6.3 Diagramme de séquence

Le diagramme de séquence détaille l'échange de messages lors d'un cycle de scan et de traitement d'une vulnérabilité :

6.4 Diagramme d'états

Le diagramme d'états illustre le cycle de vie d'une vulnérabilité, de sa découverte à sa résolution :

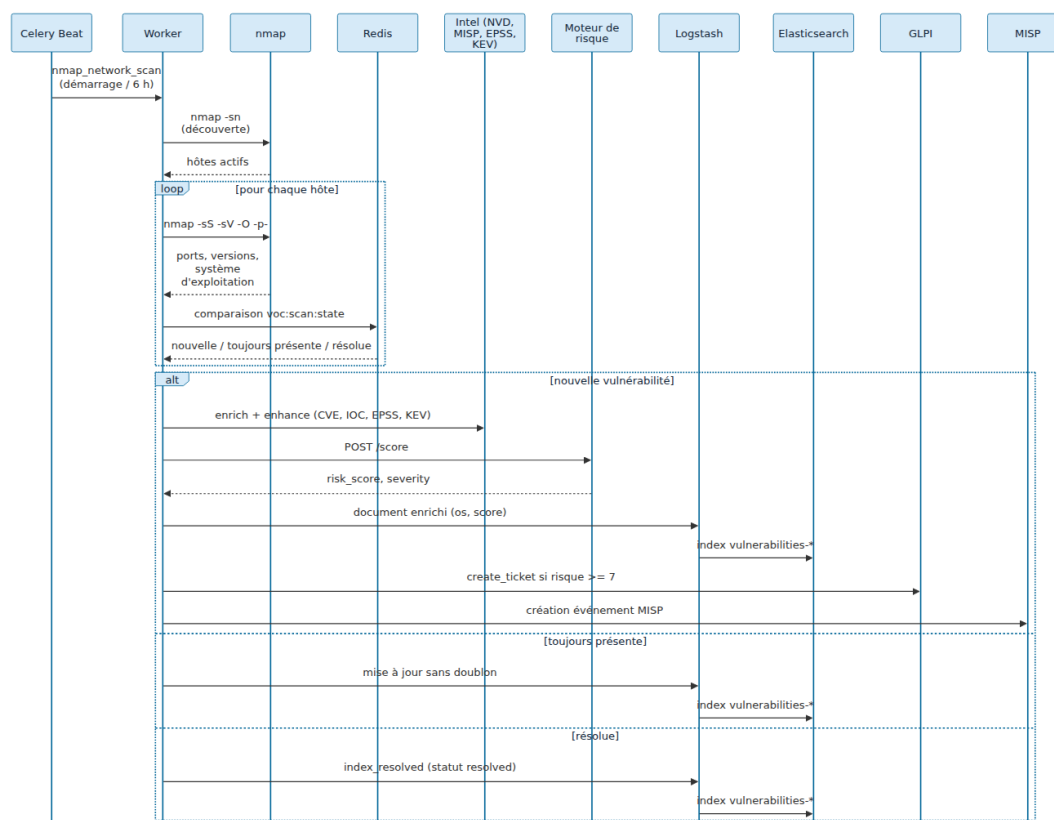


Figure 4 – Diagramme de séquence du cycle de scan et de traitement

6.5 Diagramme de déploiement

Le diagramme de déploiement présente la répartition physique des composants sur l'hôte VMware et les hôtes externes :

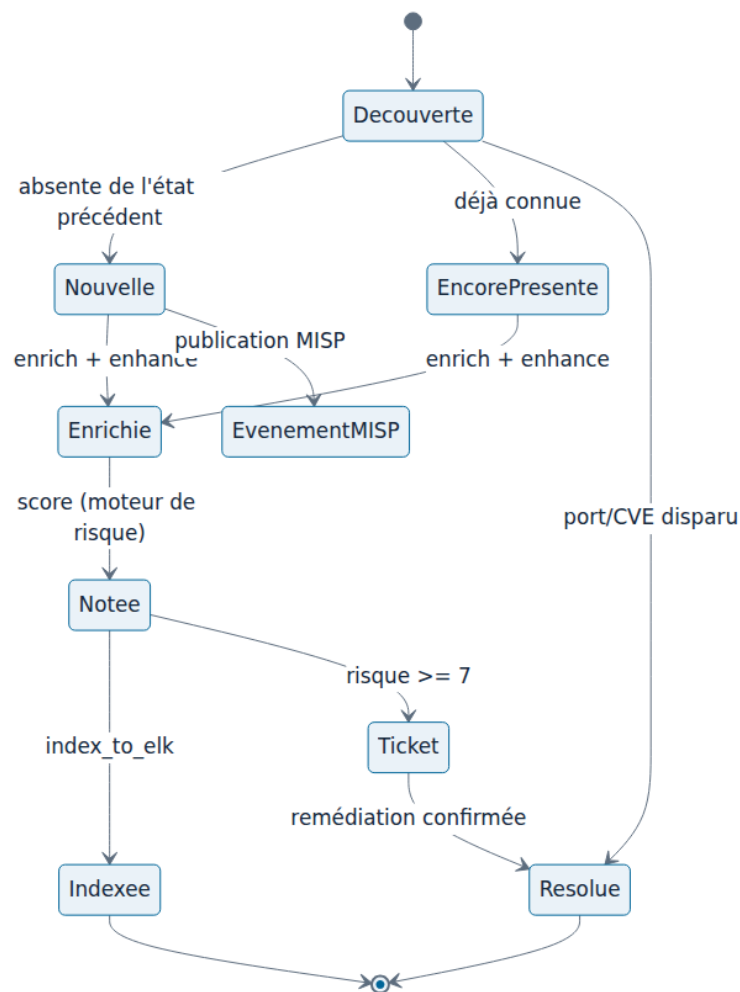


Figure 5 – Diagramme d'états du cycle de vie d'une vulnérabilité

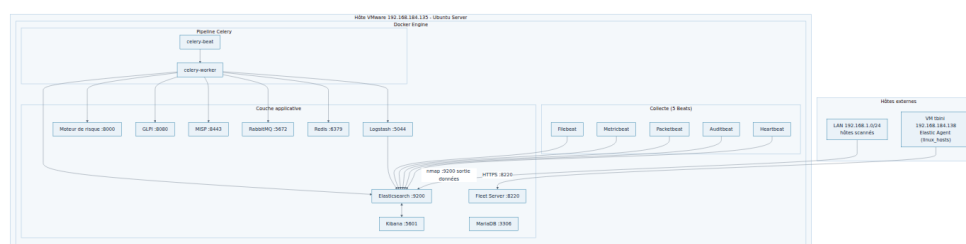


Figure 6 – Diagramme de déploiement de la plateforme

7 Pipeline de gestion des vulnérabilités

7.1 Déroulement général

Le pipeline est déclenché par Celery Beat : **au démarrage du conteneur** (via le script d'entrée `entrypoint-beat.sh`) puis **toutes les 6 heures** (`crontab hour='*/6'`). Chaque cycle passe par les étapes suivantes :

7.2 Description des étapes

Étape	Description
<code>nmap_network_scan</code>	Tâche périodique (démarrage + toutes les 6h) qui découvre les hôtes actifs avec <code>nmap -sn</code> et renvoie la liste des hôtes.
<code>scan_host</code>	Scan complet par hôte : <code>nmap -sS -sV -version-intensity 5 -O -p- -T4</code> , c'est-à-dire l'intégralité des 65535 ports TCP , la version des services et la détection du système d'exploitation (<code>osmatch</code>). Le champ <code>os</code> est transmis au document indexé.
<code>process_network_results</code>	Rappel du <i>chord</i> : compare les résultats à l'état mémorisé (<code>voc:scan:state</code> dans Redis) et classe chaque clé <code>hôte cve port</code> en <i>nouvelle</i> , <i>toujours présente</i> ou <i>résolue</i> , puis déclenche le sous-pipeline adapté.
<code>enrich</code>	Recherche MISP (<code>restSearch</code>) de la CVE et rattachement des IOC correspondants.
<code>enhance</code>	Enrichissement approfondi : EPSS, CISA KEV, OSV, Exploit-DB, VirusTotal, classification OWASP, tactiques/techniques MITRE ATT&CK, remédiation, durcissement CIS et liste de vérifications.
<code>score</code>	Appel du moteur de risque (<code>POST /score</code>) pour calculer le score 0-10 et la sévérité.
<code>index_to_elk</code>	Envoi du document enrichi (y compris le champ <code>os</code>) à Logstash (:5044), qui l'achemine vers l'indice <code>vulnerabilities-AAAA.MM.JJ</code> .
<code>create_ticket</code>	Création d'un ticket GLPI si <code>risk_score</code> $\geq$ 7, avec une checklist en suivi. La déduplication (<code>find_existing_ticket</code>) vérifie l'absence de ticket ouvert pour le même (<code>hôte</code> , <code>CVE</code>).
<code>sync_glpi_to_elk</code>	Tâche horaire qui synchronise l'état des tickets GLPI vers l'indice <code>glpi-*</code> pour les tableaux de bord.

7.3 Analyse différentielle

L'état de chaque découverte est représenté par une clé `hôte|cve|port` stockée dans Redis. À chaque cycle :

- une clé absente de l'état précédent est classée **nouvelle** (création d'un événement MISP, enrichissement complet, ticket si le risque est suffisant) ;
- une clé présente dans les deux états est classée **toujours présente** (ré-enrichissement sans doublon de ticket ou d'événement) ;
- une clé présente précédemment mais absente maintenant est classée **résolue** (`index_resolved`).

Ce mécanisme permet de mesurer l'évolution du risque dans le temps et d'éviter la duplication des tickets.

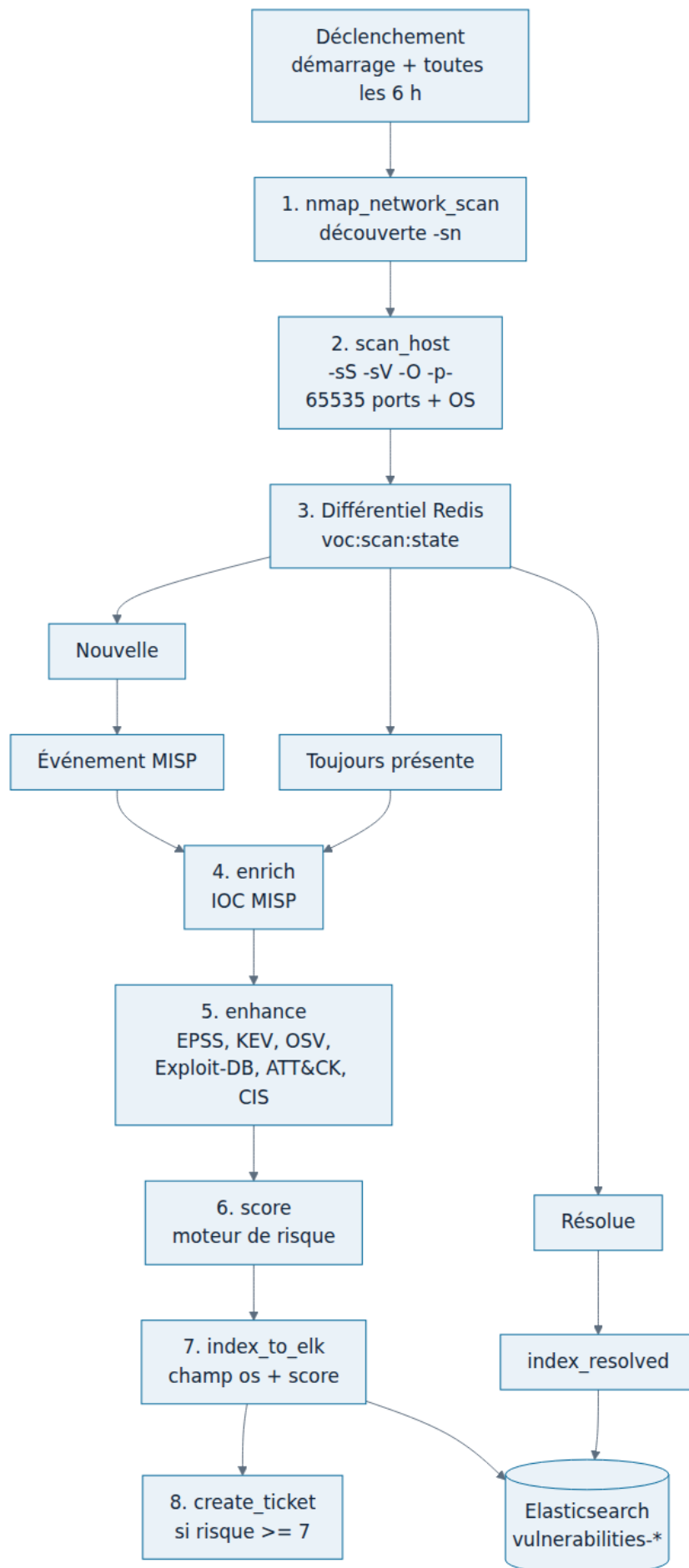


Figure 7 – Pipeline de scan, enrichissement et indexation des vulnérabilités

7.4 Synchronisation GLPI → ELK

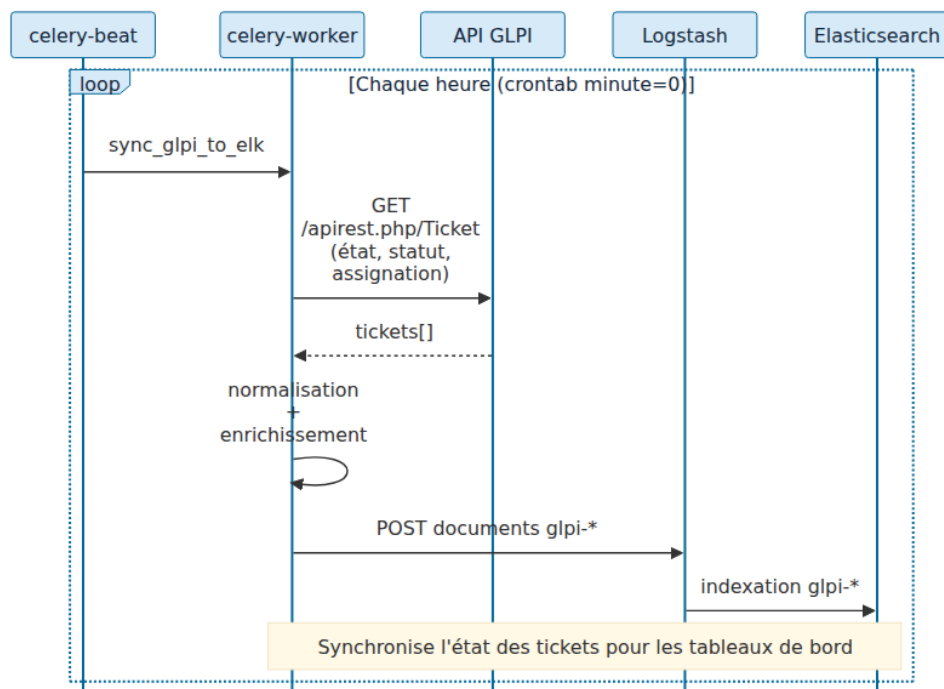


Figure 8 – Synchronisation horaire de l'état des tickets GLPI

7.5 Ordonnancement et files

Tâche	Planification	Description
nmap_network_scan	démarrage + 0 */6 * * *	Scan complet du sous-réseau + différentiel
sync_glpi_to_elk	0 * * * * (horaire)	Synchronisation des tickets GLPI vers ELK

Les tâches sont routées vers des files RabbitMQ dédiées :

Tâche	File	Rôle
discovery / scan_host	scan	Découverte et scan (nmap)
enrich / enhance	enrich	Enrichissement intelligence
score	score	Calcul du score de risque
index_to_elk	index	Envoi vers Logstash/Elasticsearch
create_ticket	ticket	Création des tickets GLPI

La configuration du worker est la suivante :

Listing 2 – Commande du worker Celery

```

1 celery -A tasks worker --loglevel=info --concurrency=2 \
2   -Q scan,enrich,score,index,ticket

```

8 Moteur de calcul du risque

Le moteur de risque est un service FastAPI sans état qui transforme la note CVSS brute en un score de risque orienté *métier* en appliquant des bonus liés au contexte.

8.1 Entrées

Champ	Intervalle	Signification
cvss_base	0–10	Note CVSS de base
is_critical_asset	booléen	L’actif est critique pour l’activité
misp_threat_active	booléen	Intelligence de menace active (MISP)
exploit_available	booléen	Un exploit public existe
network_exposure	0–1	Facteur d’exposition réseau
epss_score	0–1	Probabilité d’exploitation EPSS
in_kev	booléen	Présence dans le catalogue CISA KEV

8.2 Algorithme de notation

Condition	Bonus
Note de base	cvss_base
is_critical_asset	+ cvss_base × 0.5
misp_threat_active	+ 2.0
exploit_available	+ 1.5
network_exposure > 0	+ network_exposure × 2
in_kev	+ RISK_KEV_BOOST (2.0 par défaut)
epss_score ≥ seuil (0.5)	+ RISK_EPSS_BOOST (1.0 par défaut)
Note finale	min(score, 10)

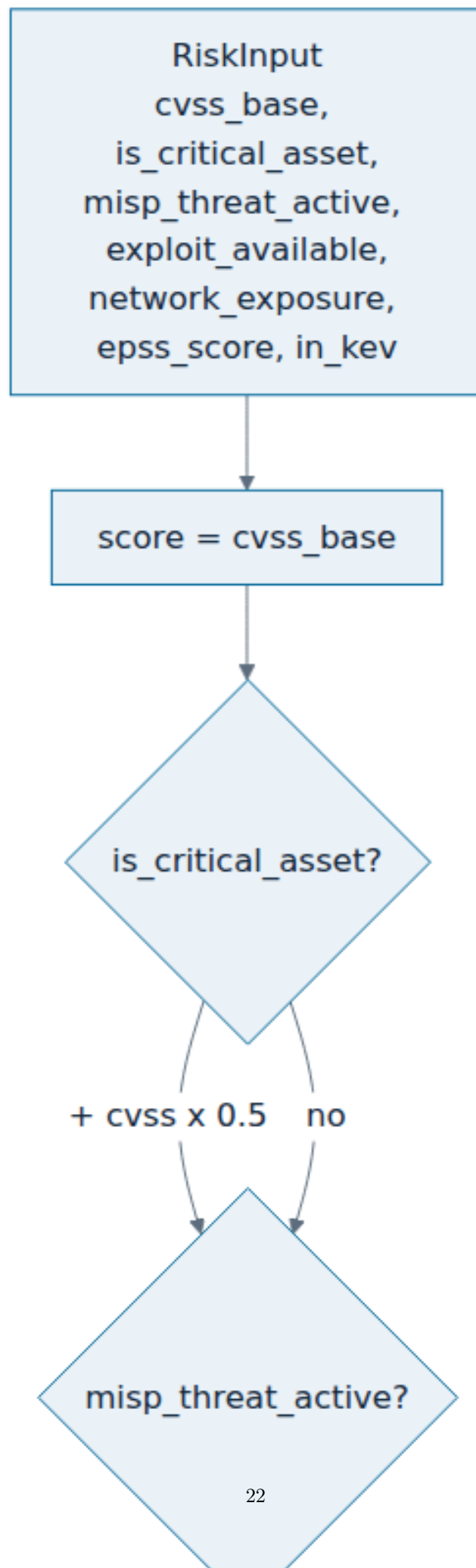
Sévérité	Intervalle de score
Critique	≥ 9
Élevée	≥ 7 et < 9
Moyenne	≥ 4 et < 7
Faible	< 4

8.3 Schéma de décision

8.4 Exemples de calcul

Exemple 1 – CVE-2024-6387 (OpenSSH *regreSSHion*) sur un actif critique, exploit public disponible, présente dans le catalogue KEV, score EPSS de 0.99 et exposition totale (= 1.0) :

Composant	Valeur
Note CVSS de base	8.1
Bonus actif critique (8.1×0.5)	+4.05
Bonus exploit public	+1.5
Bonus exposition (1.0×2)	+2.0
Bonus KEV	+2.0
Bonus EPSS ($0.99 \geq 0.5$)	+1.0
Score brut	18.65
Score final	10.0 (Critique)



Exemple 2 – Vulnérabilité CVSS 4.3 sur un actif non critique, sans exploit public, EPSS de 0.01, absente du KEV, exposition faible ($= 0.2$) :

Composant	Valeur
Note CVSS de base	4.3
Bonus exposition (0.2×2)	+0.4
Score brut	4.7
Score final	4.7 (Moyenne)

Ces deux exemples illustrent l'intérêt du modèle : une faille à fort impact réellement exploitée est portée à un niveau *Critique*, tandis qu'une faille à faible exposition reste *Moyenne*.

8.5 Règles de création des tickets GLPI

- Ticket créé uniquement si `risk_score` ≥ 7.0 .
- **Urgence** : 5 (critique) si risque ≥ 9 , sinon 4.
- **Impact** : 4 (toujours).
- **Priorité** : 5 (très élevée) si risque ≥ 7 , sinon 4.
- **Titre** : [VOC] {sévérité} - {CVE} sur {hôte}.

9 Modèle de données et stockage

9.1 Indices Elasticsearch

Modèle d'indice	Source	Contenu
vulnerabilities-AAAA.MM.JJ	Logstash	Résultats de scan avec scores de risque
filebeat-8.11.0-AAAA.MM.JJ	Filebeat	Journaux Docker, système, ICMP
packetbeat-8.11.0-AAAA.MM.JJ	Packetbeat	Métadonnées de trafic réseau
auditbeat-8.11.0-AAAA.MM.JJ	Auditbeat	Événements processus/fichiers/utilisateurs
metricbeat-8.11.0-AAAA.MM.JJ	Metricbeat	Métriques système, Docker, services
heartbeat-8.11.0-AAAA.MM.JJ	Heartbeat	Vérifications de disponibilité
glpi-AAAA.MM.JJ	Logstash	État des tickets GLPI synchronisé

9.2 Vues de données Kibana

ID	Modèle	Nom
5ec5e7c3-...	filebeat-*	VOC Filebeat Logs
8cd32128-...	packetbeat-*	VOC Network Traffic
943fb14e-...	auditbeat-*	VOC Audit Trail
metricbeat-voc	metricbeat-*	VOC Metrics
heartbeat-voc	heartbeat-*	VOC Uptime
997fc760-...	vulnerabilities-*	VOC Vulnerabilities
-	glpi-*	GLPI Tickets

9.3 Exemple de document de vulnérabilité

Un document indexé via Logstash contient les champs suivants :

Listing 3 – Document de vulnérabilité indexé (abrégé)

```

1 {
2   "@timestamp": "2026-08-15T04:57:00Z",
3   "host_ip": "192.168.1.1",
4   "os": "Netgear WGR614v7",
5   "cve": "CVE-2017-9765",
6   "cvss": 9.0,
7   "risk_score": 10.0,
8   "severity": "Critical",
9   "port": "7547/tcp",
10  "source": "voc-nmap",
11  "status": "active",
12  "epss_score": 0.80,
13  "in_kev": true,
14  "exploit_available": true,
15  "vulnerability_type": "Security Vulnerability",
16  "cwe_ids": ["CWE-78"],
17  "attack_tactics": ["Initial Access"],
18  "attack_techniques": ["T1190"],
19  "checklist": [ { "id": "VAL-01", "category": "Validation", "done": false
20                  } ],
21  "technical_description": "## Vulnerability Overview ..."

```

Le champ `os` est désormais présent grâce à la détection du système d'exploitation, et le champ `risk_score` alimente les tableaux de bord et les règles de tickets.

10 Sécurité de la plateforme

10.1 Chiffrement de Kibana

Kibana nécessite des clés de chiffrement pour protéger les objets sauvegardés, les sessions et les rapports. Trois clés ont été injectées via les variables d'environnement suivantes :

Variable	Rôle
XPACK_SECURITY_ENCRYPTIONKEY	Chiffrement des sessions et données sécurisées
XPACK_ENCRYPTEDSAVEDOBJECTS_ENCRYPTIONKEY	Chiffrement des objets sauvegardés
XPACK_REPORTING_ENCRYPTIONKEY	Chiffrement des rapports

10.2 Authentification

Chaque service dispose de mécanismes d'authentification : utilisateur `elastic` pour Elasticsearch/-Kibana, jetons d'application et d'utilisateur pour l'API GLPI, et clé API pour MISP. Les secrets sont stockés dans le fichier `.env` et injectés dans les conteneurs.

10.3 Isolation des conteneurs

La plateforme utilise des réseaux et volumes Docker dédiés. Le worker de scan exécute `nmap` en tant que `root` avec la capacité `NET_RAW` pour permettre le scan SYN (`-sS`) et la détection OS (`-O`) ; cette exécution privilégiée est limitée au seul conteneur de scan. Les limites mémoire sont définies par service afin d'éviter qu'un conteneur n'épuise les ressources de l'hôte.

10.4 Supervision des agents

Fleet centralise l'enrôlement des agents et la distribution des politiques. L'état de chaque agent (`online`, `offline`, `degraded`) est visible depuis Kibana, ce qui permet de détecter rapidement les agents qui ne transmettent plus de données.

10.5 Transport des données

En environnement de laboratoire, les certificats autogénérés sont utilisés. Les clients (MISP, GLPI, Elasticsearch) acceptent une vérification SSL désactivable via des variables d'environnement (`MISP_VERIFY_SSL`, `GLPI_VERIFY_SSL`) afin de permettre un durcissement lors d'un passage en production.

11 État opérationnel du déploiement

11.1 Santé de la plateforme

La plateforme est entièrement fonctionnelle : les 20 services Docker (18 applicatifs + serveur Fleet + tâche d'initialisation ponctuelle) sont déployés et, pour la quasi-totalité, *healthy*.

Indicateur	Valeur
Conteneurs Docker	20 déployés (18 + Fleet Server + initialisation)
Cluster Elasticsearch	1 nœud, statut green
Shards	129 primaires actifs, 0 non-assignés
Documents de vulnérabilités (total)	5,041
Documents vulnérabilités du jour (08.15)	2,992
Hôtes analysés	4 (dont routeur 192.168.1.1)
Hôtes actifs découverts (LAN)	8 (routeurs, WAP, postes, serveurs)
Tickets GLPI	419 (statut = nouveau, avec checklists)
Événements MISP	1,300+

11.2 Preuves de l'extension du scan

Le passage au scan complet a été vérifié en production :

- le scan démarre **au démarrage du conteneur** puis toutes les 6 h (exécution vérifiée) ;
- **détection du système d'exploitation** opérationnelle : ex. Linux 5.0 - 5.14 pour 192.168.1.18, et lors de la découverte : routeur Netgear WGR614, point d'accès TP-LINK TD-W8963N, poste Windows 11 et smartphones Android ;
- le port 7547/tcp (TR-069, hors de l'ancienne liste de 41 ports) a été découvert, preuve que **l'intégralité des ports** est maintenant scannée ; il a déclenché les tickets GLPI #418 et #419 (critiques, hôte 192.168.1.1) ;
- l'analyse différentielle du dernier cycle : **2 nouvelles** / 307 toujours présentes / 0 résolue ;
- deux vulnérabilités à risque 10.0 ont été indexées et un événement MISP a été créé ;
- le champ `os` est bien présent dans les documents Elasticsearch.

11.3 Répartition des données (indice du 08.15)

Hôte	Documents	Observations
192.168.1.18	1,778	serveur LAN (port 3306/MySQL majoritaire)
192.168.184.135	1,119	hôte plateforme (OpenSSH, services Docker)
192.168.184.136	93	second hôte du réseau SPA
192.168.1.1	2	routeur découvert via le scan complet

11.4 Ports les plus fréquents

Port	Documents	Observation
3306/tcp	2,400	MySQL/MariaDB (hôte .18)
22/tcp	323	SSH (serveurs)
8080/tcp	252	HTTP alternatif
9200/tcp	15	Elasticsearch exposé
7547/tcp	2	TR-069 (routeur) – nouvellement découvert

11.5 Agents Fleet

Agent	Hôte	Politique	État
2fbcef12-...	voc-fleet-server	fleet-server-policy-fresh-001	online
607d470b-...	conteneur élastique	linux_hosts	online
f7f2c317-...	tbini (VM 192.168.184.138)	linux_hosts	degraded

12 Difficultés rencontrées et solutions apportées

12.1 Serveur Fleet instable

Le serveur Fleet rencontrait des arrêts répétés (*OOM-kill*) et perdait son identité d'agent à chaque redémarrage. La solution a consisté à : limiter sa mémoire (`mem_limit: 1g`), l'exécuter en tant que `root` et monter un volume persistant (`fleet_server_state`) afin de conserver le fichier d'état entre deux redémarrages. Le serveur est désormais stable et son agent *online*.

12.2 Erreurs de chiffrement de Kibana

Au démarrage, Kibana émettait des erreurs sur les clés de chiffrement (`encryptedSavedObjects`, `security`, `reporting`). L'étude de l'entrypoint officiel a montré qu'il mappe les variables d'environnement *par nom*, sans découper la casse camelCase. Les variables ont donc été renommées (`XPACK_ENCRYPTEDSAVEDOBJECTS_ENCRYPTIONKEY`, `XPACK_SECURITY_ENCRYPTIONKEY`, `XPACK_REPORTING_ENCRYPTIONKEY`) et injectées à la fois dans le fichier de configuration et dans Docker Compose, ce qui a résolu le problème.

12.3 Agent externe sans flux de données

Un agent enrôlé sur une VM externe (`tbini`, `192.168.184.138`) se connectait bien au plan de contrôle (`:8220`) mais restait *degraded* sans produire de données. La cause identifiée : la sortie de données par défaut de Fleet pointe vers `http://elasticsearch:9200`, un nom DNS interne à Docker que les agents externes ne peuvent pas résoudre. La correction consiste à faire pointer la sortie par défaut vers l'adresse réseau réelle (`http://192.168.184.135:9200`), compatible avec les agents hébergés hors conteneurs.

12.4 Stabilité du worker

Le worker Celery était victime d'OOM (*SIGKILL*) en maintenant plusieurs chaînes de centaines de découvertes avec une concurrence de 4, provoquant des boucles de re-livraison. La concurrence a été abaissée à 2, les limites mémoire du worker et de Redis relevées, et les tâches bloquées purgées.

12.5 Santé d'Elasticsearch

Le cluster monopôle était *yellow* avec des réplicas non assignés. Un *index template* (`voc-beats-zero-replicas`) a été créé pour forcer 0 réplica sur les indices des Beats ; le cluster est désormais *green*.

12.6 Scan partiel

L'ancienne liste fixe de 41 ports manquait des services exposés sur d'autres ports (ex. `7547`). Le scan est passé à l'intégralité des ports (`-p-`) avec détection OS (`-O`), au démarrage puis toutes les 6 h.

12.7 Tickets dupliqués

Les scans répétés généraient des tickets GLPI en double. La fonction `find_existing_ticket` vérifie désormais l'existence d'un ticket ouvert pour le couple (`hôte`, `CVE`) avant toute création.

13 Tests et validation

13.1 Tests unitaires

Le dépôt contient des tests unitaires couvrant trois pôles :

- **Intelligence sur les menaces** : parsing des réponses EPSS et KEV, cache, recherche OSV et Exploit-DB ;
- **Enrichissement des tickets** : classification OWASP, correspondances MITRE ATT&CK, génération des checklists ;
- **Moteur de risque** : validation des entrées et des scores, bornes de sévérité, effets des bonus.

Ils sont exécutés dans les conteneurs du worker et du moteur de risque :

Listing 4 – Exécution des tests

```
1 docker exec voc-celery-worker python -m pytest /app/tests -v
2 docker exec voc-risk-engine    python -m pytest /app/tests -v
```

13.2 Validation de bout en bout

La chaîne complète a été validée en conditions réelles :

1. déclenchement du scan au démarrage du conteneur ;
2. découverte de 8 hôtes actifs sur le réseau LAN ;
3. scan complet avec détection OS et découverte du port 7547 ;
4. création de l'événement MISP, indexation des documents dans Elasticsearch, ouverture des tickets GLPI #418 et #419 ;
5. vérification du champ `os` et du score de risque dans les documents indexés.

14 Conclusion générale

Le stage effectué au sein de la société **Gérence Informatique**, encadré par **Mr. MDAOUKHI Adel**, m'a permis de concevoir, déployer et d'exploiter la **VOC Platform**, une plateforme complète de gestion automatisée des vulnérabilités.

La plateforme répond à la problématique posée : elle scanne automatiquement le réseau (tous les ports, avec détection des systèmes d'exploitation), corrèle les découvertes avec plusieurs sources d'intelligence sur les menaces (NVD, MISP, EPSS, CISA KEV, OSV, Exploit-DB), calcule un score de risque orienté métier, publie les résultats dans des tableaux de bord Elastic et ouvre automatiquement des tickets de remédiation dans GLPI. Elle est désormais opérationnelle : cluster Elasticsearch *green*, plus de 5 000 documents de vulnérabilités indexés, 419 tickets de remédiation créés, un partage de renseignements actif via MISP et une gestion centralisée des agents via Fleet.

Sur le plan technique, ce projet m'a permis de mettre en pratique les conteneurs Docker, l'architecture de messages (RabbitMQ), l'ordonnancement de tâches (Celery), la stack Elastic et la conception d'une API avec FastAPI. Les difficultés rencontrées (stabilité de Fleet, clés de chiffrement de Kibana, agents externes, montée en charge du worker) ont été systématiquement analysées et résolues, ce qui a été très formateur.

Enfin, ce stage m'a également permis de développer des compétences transversales : analyse de problèmes d'exploitation, documentation technique, travail en autonomie et communication avec l'équipe. La plateforme reste évolutive : elle pourra être étendue à d'autres réseaux, à des scans authentifiés plus profonds et à l'enrôlement d'agents supplémentaires pour enrichir encore la supervision sécuritaire de l'infrastructure.

A Annexe A : variables d'environnement principales

Variable	Défaut	Rôle
BROKER_URL	amqp://voc:...@rabbitmq:5672//	Courtier Celery
RESULT_BACKEND	redis://:...@redis:6379/0	Backend Celery
DISCOVERY_SUBNET	192.168.184.0/24	Cible du scan nmap
NMAP_SCAN_ARGS	-sS -sV -version-intensity 5 -O -p- -T4	Arguments de scan par défaut
MISP_URL	https://192.168.184.135:8443	Base API MISP
GLPI_URL	http://192.168.184.135:8080	Base API GLPI
RISK_ENGINE_URL	http://risk-engine:8000	Service de risque
LOGSTASH_URL	http://logstash:5044	Ingestion ELK
RISK_KEY_BOOST	2.0	Bonus KEV
RISK_EPSS_THRESHOLD	0.5	Seuil du bonus EPSS
RISK_EPSS_BOOST	1.0	Bonus EPSS
XPack_SECURITY_ENCRYPTIONKEY	-	Chiffrement sécurité Kibana
XPack_ENCRYPTEDSAVEDOBJECTS_ENCRYPTIONKEY	-	Chiffrement des objets Kibana
XPack_REPORTING_ENCRYPTIONKEY	-	Chiffrement des rapports
CHECKLIST_IN_GLPI	true	Checklist dans les tickets

B Annexe B : opérations courantes

Listing 5 – Opérations courantes

```

1 # Sante du cluster
2 curl -u elastic:\$ELASTIC_PASSWORD http://localhost:9200/_cluster/health
3
4 # Scan manuel du sous-reseau
5 docker exec voc-celery-worker python -c \
6     "from tasks import nmap_network_scan; nmap_network_scan.delay
7     ('192.168.184.0/24')"
8
9 # Sante du moteur de risque
10 curl http://localhost:8000/health
11
12 # Noter un exemple de decouverte
13 curl -X POST http://localhost:8000/score -H 'Content-Type: application/
14     json' \
15     -d '{"cvss_base": 7.5, "is_critical_asset": true, "exploit_available":
16     true}'
17
18 # Journaux du worker
19 docker logs -f voc-celery-worker

```

C Annexe C : tests

Listing 6 – Exécution des tests

```

1 docker exec voc-celery-worker python -m pytest /app/tests -v
2 docker exec voc-risk-engine python -m pytest /app/tests -v

```

D Annexe D : glossaire

Terme	Définition
CVE	<i>Common Vulnerabilities and Exposures</i> – identifiant public d’une vulnérabilité
CVSS	<i>Common Vulnerability Scoring System</i> – note de gravité 0–10
EPSS	<i>Exploit Prediction Scoring System</i> – probabilité d’exploitation
KEV	<i>Known Exploited Vulnerabilities</i> – catalogue CISA des failles exploitées
IOC	<i>Indicator of Compromise</i> – indicateur de compromission
ITSM	<i>IT Service Management</i> – gestion des services informatiques
SOC	<i>Security Operations Center</i> – centre d’opérations de sécurité
TIP	<i>Threat Intelligence Platform</i> – plateforme d’intelligence sur les menaces
AMQP	<i>Advanced Message Queuing Protocol</i> – protocole de messagerie
OS	<i>Operating System</i> – système d’exploitation